Week 4 · Lab 2

Version B

Graded Practice:

Operators and `if/else`

10 problems from LeetCode · 10 points each · checked by the TA at your desk

Week: 4 — second session (graded)
Duration: 2 hours 15 minutes
Format: individual problem solving in CLion
Covers: Lecture 3 — operators, short-circuit, `if/else if`, nested `if`, output formatting (`<iomanip>`)

Name:

Group:

Date:

How this lab works

1. Solve the problems in CLion, one `.cpp` file per problem. Every program reads its input with `cin` and prints its answer with `cout`.
2. **When a problem works, raise your hand.** The TA comes to your desk, reads your code, runs a few inputs and writes your points in the box under the problem. Nothing is uploaded.
3. There are **10 problems, 10 points each** (100 in total): problems 1–4 are **easy**, 5–7 **medium** and 8–10 **hard**. Solve them in any order; a problem with no attempt earns 0 points.
4. Under each title, **Practises:** says which part of Lecture 3 the problem trains.
5. You may ask the TA for help — this is practice, not an exam. The TA may ask you a short question about your code; be ready to explain *why* it works.

All problems are from `leetcode.com`, restated for a program that reads from the keyboard (some are restricted to a fixed number of inputs, because loops and arrays come later). On LeetCode itself the answer is written as a *function* inside `class Solution`; you will learn functions in Week 6.

Quick reference (Lecture 3)

Arithmetic `17 / 5` is 3 and `17 % 5` is 2 (`int / int` throws the fraction away); `-7 / 2` is `-3`. `n % 10` is the last digit, `n / 10 % 10` the tens digit. Convert *before* dividing: `static_cast<double>(a) / b`. Compound forms: `x += 5; x %= 3;` `x++` / `++x` add 1.

Precedence, highest first: `()` · `!` · `* / %` · `+ -` · `< <= > >=` · `== !=` · `&&` · `||` · `=`. Inside `cout`, put a comparison in parentheses: `cout << (a < b);`

Conditions A range is two tests: `3 < x && x < 5` (`3 < x < 5` compiles but is always true). `&&` stops at the first false, `||` at the first true, so put the guard on the left:

```
if (c != 0 && a / c > 1) { ... }   // never divides by 0
```

Give long conditions a name, and remember that in arithmetic `true` counts as 1 and `false` as 0:

```
bool leap = (y % 4 == 0 && y % 100 != 0) || y % 400 == 0;
int odd = (a % 2 == 1) + (b % 2 == 1);   // 0, 1 or 2
```

Reading formatted input `cin >> c` for a `char` skips spaces and reads one character, so separators go into a spare `char`:

```
int y, m, d;  char dash;
cin >> y >> dash >> m >> dash >> d;     // 2019-02-10
```

`else if` chains The *first* true condition wins and the rest are skipped — test the strictest case first.

```
if (avg >= 90) {
    cout << "Excellent";
} else if (avg >= 75) {      // here avg < 90 already
    cout << "Good";
} else {
    cout << "Needs work";
}
```

Nested `if` An `else` binds to the *nearest* unmatched `if`. Always use braces.

Maximum, minimum, absolute value

```
int mx = a;
if (b > mx) { mx = b; }
if (c > mx) { mx = c; }      // mx = max(a, b, c)
int d = a - b;
if (d < 0) { d = -d; }       // d = |a - b|
```

Formatting output `#include <iomanip>`

```
cout << fixed << setprecision(2) << 2.5;   // 2.50
cout << setfill('0') << setw(3) << 7;      // 007
```

`fixed`, `setprecision` and `setfill` stay in force; `setw` applies to the *next item only*. Without `fixed`, `setprecision(2)` means 2 *significant* digits.

Classic bugs `if (x = 5)` (use `==`); `if (x > 0);` (stray `;`); `x == 6 || 28` (always true — write `x == 6 || x == 28`); `int` overflow (use `long long`).

EASY

Problems 1–4 · 40 points

Short programs: one idea each. Test every branch, not just the examples.

1 Find Three Consecutive Integers That Sum to a Given Number 10 pts

LeetCode 2177 · `leetcode.com/problems/find-three-consecutive-integers-that-sum-to-a-given-number`

Practises: `if/else` — and choose the type carefully.

Read `num` ($0 \le$ `num` $\le 10^{15}$). If `num` is the sum of three consecutive integers, print them, smallest first; otherwise print `none`.

Input	Output	Why
`33`	`10 11 12`	$10 + 11 + 12 = 33$.
`4`	`none`	

Hint: $(x - 1) + x + (x + 1) = 3x$. *Check the size of* `num` *before choosing its type.*

TA: ____ / 10 pts initials ____

2 Power of Three 10 pts

LeetCode 326 · `leetcode.com/problems/power-of-three`

Practises: short-circuit && as a guard.

Read an integer n (any `int`, negative values included). Print `true` if n is a power of three, that is $n = 3^x$ for some whole number $x \ge 0$; otherwise print `false`.

Input	Output	Why
`27`	`true`	3^3.
`0`	`false`	
`-1`	`false`	

Hint: $3^{19} = 1162261467$ *is the largest power of three that fits in an* `int`. *Because 3 is prime, the only divisors of* 3^{19} *are powers of three. Be careful with* $n = 0$ *and negative* n.

TA: ____ / 10 pts initials ____

3 Robot Return to Origin (4 moves) 10 pts

LeetCode 657 · `leetcode.com/problems/robot-return-to-origin`

Practises: `char` comparisons, and `bool` values as 0/1.

A robot starts at $(0, 0)$. Read exactly **four** moves written together, for example `UDRL`, where `U` is up, `D` down, `L` left and `R` right (one step each). Print `true` if the robot ends at $(0, 0)$, otherwise `false`. (LeetCode allows any number of moves; here it is always four.)

Input	Output	Why
`UDRL`	`true`	Up and down cancel, right and left cancel.
`LLUR`	`false`	It ends at $(-1, 1)$.

Hint: `cin >> c1 >> c2 >> c3 >> c4;` *reads the four letters one by one. The robot is back if it went up as often as down and left as often as right. In arithmetic a comparison counts as 1 or 0, so* `(c1 == 'U') + (c2 == 'U') + ...` *counts the* `U`*s.*

TA: ____ / 10 pts initials ____

4 Divisor Game 10 pts

LeetCode 1025 · `leetcode.com/problems/divisor-game`

Practises: `if/else` — the work is in the reasoning.

Alice and Bob take turns, Alice first. A number n is on the board. On a turn, the player picks some x with $0 < x < n$ and $n \% x = 0$, and replaces n by $n - x$. A player who cannot move loses. Both play perfectly. Read n ($1 \le n \le 1000$) and print `true` if Alice wins, otherwise `false`.

Input	Output	Why
`2`	`true`	Alice picks 1; Bob faces 1 and cannot move.
`3`	`false`	Alice must pick 1; Bob picks 1; Alice faces 1.

Hint: play $n = 1, 2, \ldots, 6$ on paper and look at who wins.

TA: _____ / 10 pts initials _____

MEDIUM

Problems 5–7 · 30 points

A few cases to get right. Test the boundaries.

5 Calculate Amount Paid in Taxes 10 pts

LeetCode 2303 · `leetcode.com/problems/calculate-amount-paid-in-taxes`

Practises: an `else if` chain.

A country has three tax brackets. Read `upper1 percent1 upper2 percent2 upper3 percent3 income`, where $0 <$ `upper1` $<$ `upper2` $<$ `upper3` ≤ 1000, each percent is from 0 to 100, and $0 \le$ `income` $\le$ `upper3`. The first `upper1` dollars are taxed at `percent1` %, the next `upper2 - upper1` dollars at `percent2` %, and the rest at `percent3` %. Print the tax with exactly two digits after the decimal point.

Input	Output	Why
`3 50 7 10 12 25 10`	`2.65`	$3 \cdot 50\% + 4 \cdot 10\% + 3 \cdot 25\% = 1.50 + 0.40 + 0.75$.
`1 0 4 25 5 50 2`	`0.25`	$1 \cdot 0\% + 1 \cdot 25\%$.

Hint: an `else if` *chain on* `income`*. Which brackets does an income of 5 reach in the first example? For the output use* `fixed` *and* `setprecision(2)` *from* `<iomanip>`*.*

TA: ____ / 10 pts initials _____

6 Maximum Score From Removing Stones 10 pts

LeetCode 1753 · `leetcode.com/problems/maximum-score-from-removing-stones`

Practises: a maximum with `if`, then `if/else`.

There are three piles of a, b and c stones ($1 \le$ each $\le 10^5$). On each turn you take one stone from two *different* non-empty piles and score 1 point. The game ends when fewer than two piles have stones. Print the maximum score.

Input	Output	Why
`2 4 6`	`6`	
`4 4 6`	`7`	
`1 8 8`	`8`	

Hint: if the biggest pile has at least as many stones as the other two together, every move must use it. If not, you can empty (almost) everything — how many moves is that?

TA: ____ / 10 pts initials _____

7 Evaluate Reverse Polish Notation 10 pts

LeetCode 150 · `leetcode.com/problems/evaluate-reverse-polish-notation`

Practises: an `else if` chain on a `char`.

In Reverse Polish Notation the operator comes *after* its two operands: `2 1 +` means 2 + 1. Read exactly five tokens `a b op1 c op2`, which mean `(a op1 b) op2 c`. The numbers are integers from −200 to 200, each operator is one of `+ - * /`, division truncates towards zero, and there is no division by zero. Print the result.

Input	Output	Why
`2 1 + 3 *`	`9`	(2 + 1) · 3.
`15 4 / 2 -`	`1`	15/4 = 3, then 3 − 2.
`-7 2 / 1 +`	`-2`	−7/2 = −3 (towards zero), then −3 + 1.

Hint: read the operators into `char`*s and compare them:* `op1 == '+'`*. Two* `else if` *chains, one per operator.*

TA: _____ / 10 pts initials _____

HARD

Problems 8–10 · 30 points

More cases and more care. Read the hint before you start.

8 Number of Days Between Two Dates 10 pts

LeetCode 1360 · `leetcode.com/problems/number-of-days-between-two-dates`

Practises: `else if` chains and a `bool` for the leap year.

Read two valid dates written `YYYY-MM-DD` (years 1971 to 2100) and print the number of days between them. A year is a leap year if it is divisible by 4 but not by 100, or if it is divisible by 400.

Input	Output	Why
`2019-06-29 2019-06-30`	`1`	
`2020-01-15 2019-12-31`	`15`	The second date may be the earlier one.

Hint: for each date count the days since 1 January 1971: 365 per year from 1971 to $y-1$, plus one per leap year among them (from 1 to p there are `p/4 - p/100 + p/400` *leap years; from 1 to 1970 there are 477), plus the day of the day of the year. Day of the year: an* `else if` *chain on the month gives the days before it in a normal year (0, 31, 59, 90, . . . , 334), plus one if the year is a leap year and $m > 2$, plus the day.*

TA: ____ / 10 pts initials _____

9 Valid Square 10 pts

LeetCode 593 · `leetcode.com/problems/valid-square`

Practises: a minimum with `if`, and `bool` values as 0/1.

Read four points `x1 y1 x2 y2 x3 y3 x4 y4` (in any order, coordinates from -10^4 to 10^4). Print `true` if they are the corners of a square with positive side length (it may be rotated), otherwise `false`.

Input	Output	Why
`0 0 1 1 1 0 0 1`	`true`	
`0 0 1 1 1 0 0 12`	`false`	
`1 0 -1 0 0 1 0 -1`	`true`	A square turned by 45°.

Hint: compute the six squared *distances between pairs of points. In a square the four sides are the smallest and all equal to some $s > 0$, and both diagonals equal $2s$. In arithmetic* `true` *counts as 1, so* `(d1 == s) + (d2 == s) + ...` *counts how many distances equal s.*

TA: ____ / 10 pts initials _____

10 Clumsy Factorial 10 pts

LeetCode 1006 · `leetcode.com/problems/clumsy-factorial`

Practises: an `else if` chain on `n % 4`.

The *clumsy factorial* of n uses the operations `*`, `/`, `+`, `-` in turn on $n, n-1, \ldots, 1$:

$$\text{clumsy}(n) = n \cdot (n-1) \,/\, (n-2) + (n-3) - (n-4) \cdot (n-5) \,/\, (n-6) + (n-7) - \ldots$$

with the usual precedence (`*` and `/` first, left to right) and integer division. Read n ($1 \le n \le 10^4$) and print clumsy(n).

Input	Output	Why
`4`	`7`	$4 \cdot 3/2 + 1 = 6 + 1$.
`10`	`12`	$10 \cdot 9/8 + 7 - 6 \cdot 5/4 + 3 - 2 \cdot 1 = 11 + 7 - 7 + 3 - 2$.

Hint: no loop is needed. Compute clumsy(5), clumsy(6), . . . , clumsy(12) by hand and compare each with n: the difference depends only on `n % 4`. *Handle $n \le 4$ separately.*

TA: _____ / 10 pts initials _____

Score sheet — Version B

	Problem	Level	Points	Earned	TA
1	Find Three Consecutive Integers That Sum to a Given Number (LC 2177)	Easy	10		
2	Power of Three (LC 326)	Easy	10		
3	Robot Return to Origin (4 moves) (LC 657)	Easy	10		
4	Divisor Game (LC 1025)	Easy	10		
5	Calculate Amount Paid in Taxes (LC 2303)	Medium	10		
6	Maximum Score From Removing Stones (LC 1753)	Medium	10		
7	Evaluate Reverse Polish Notation (LC 150)	Medium	10		
8	Number of Days Between Two Dates (LC 1360)	Hard	10		
9	Valid Square (LC 593)	Hard	10		
10	Clumsy Factorial (LC 1006)	Hard	10		
	Total		**100**		

After the lab

Every problem here is on LeetCode under the number shown. Some were cut down to a fixed number of inputs; with the loops of Week 5 you can solve the full versions. Once functions are covered (Week 6), rewrite your solutions as a LeetCode `class Solution` and submit them.

End of Week 4 · Lab 2 · Version B